

Protocols and Behaviours — A Readable Companion to the Elixir Docs

Chapter 8 · checked against Elixir 1.20 · examples run on Elixir 1.14 and 1.18 · Sep 26, 2026

1. Why Elixir has two kinds of polymorphism

Elixir has no classes, but it has two ways to write code that works across different things:

- A **protocol** dispatches on the *data type* of the first argument. `Enumerable`, `String.Chars` (`to_string/1`) and `Inspect` are protocols: `Enum.map/2` works on lists, maps, ranges and your own structs because each has an implementation. Anyone can add an implementation for a new type, without touching the protocol.
- A **behaviour** is a contract for a *module*: a list of callbacks the module promises to define. `GenServer`, `Supervisor` and `Application` are behaviours; you pick the module explicitly (`GenServer.start_link(MyServer, ...)`), and the behaviour's generic code calls your callbacks.

The short rule: vary by **data**, use a protocol; vary by **implementation you choose**, use a behaviour (plus configuration or dependency injection).

2. The building blocks

Protocols

```
defprotocol Shape do
  @spec area(t) :: number
  def area(shape)
end

defimpl Shape, for: Circle do
  def area(%Circle{r: r}), do: :math.pi() * r * r
end
```

Feature	Meaning
<code>defimpl P, for: Type</code>	Implementation for a built-in type or a struct
<code>@derive [P] / @derive {P, opts}</code>	Generate an implementation for a struct from the protocol's <code>Any</code> implementation
<code>@fallback_to_any true</code>	Use the <code>Any</code> implementation for every type without its own (the docs prefer explicit <code>@derive</code>)
<code>Protocol.UndefinedError</code>	Raised when a type has no implementation
Consolidation	Mix compiles all implementations into fast dispatch at build time

Structs do not inherit map implementations: a struct is not `Enumerable` and does not support `Access` unless you implement them.

Behaviours

```
defmodule Notifier do
  @callback deliver(user :: map, message :: String.t()) :: :ok | {:error, term}
end

defmodule EmailNotifier do
  @behaviour Notifier
  @impl true
  def deliver(user, msg), do: ...
end
```

Feature	Meaning
@callback	Declares a required function and its typespec
@behaviour Mod	Declares that this module implements it; missing callbacks produce warnings
@impl true	Marks a function as a callback; the compiler checks it matches one
@optional_callbacks	Callbacks a module may skip
use Mod	Often sets @behaviour and injects default implementations

3. How it actually works

use injects defaults, so a typo is never missing

`use GenServer` defines default versions of callbacks you don't write. If you misspell one (`handle_cal/3`), your function is just an ordinary extra function, and the default handles the real callback: `GenServer.call/2` fails at runtime with "no `handle_call/3` clause was provided" (verified). Nothing warns at compile time, unless you mark callbacks with `@impl true`. Then the typo produces a warning: "got `@impl true` for function `handle_cal/3` but no behaviour specifies such callback" (verified).

Inspect shows everything by default

The default `Inspect` implementation for a struct prints every field. Logs, crash reports, `IO.inspect` and error trackers all use it. Structs holding secrets need an explicit `@derive {Inspect, only: ...}` or `except: ...`.

Dispatch is on the first argument only

A protocol picks its implementation from the type of the first argument, nothing else. For behaviour that depends on two types, or on configuration, a protocol is the wrong tool.

4. Plausible but wrong

4.1 Secrets in logs

```
defmodule Account do
  defstruct [:email, :password_hash, :api_token]
end

Logger.error("sync failed for #{inspect(account)}")
```

Default `inspect` prints `password_hash` and `api_token` into the log, and from there into log storage, error trackers and support tickets (verified). Fix: `@derive {Inspect, except: [:password_hash, :api_token]}`, which prints `#Account<email: "a@b.c", ...>` (verified). Better still, use `only:` to list the fields that are safe.

4.2 A misspelled callback

```
defmodule Pinger do
  use GenServer
  def init(s), do: {:ok, s}
  def handle_cal(:ping, _from, s), do: {:reply, :pong, s}
end
```

It compiles without a word. The first `GenServer.call(pid, :ping)` crashes the server with “no `handle_call/3` clause was provided”. Fix: put `@impl true` on every callback, so a misspelled name produces a compile-time warning. Once one callback in a module has `@impl`, the compiler also warns about callbacks that lack it, which keeps the habit consistent.

4.3 Enum over a struct

```
def total(%Order{} = order) do
  Enum.reduce(order, 0, fn {_k, v}, acc -> acc + v end)
end
```

Structs are not `Enumerable`, so this raises `Protocol.UndefinedError` (verified). A test with a plain map passes. Fix: iterate over the field you mean (`order.items`). If you really need the fields, use `Map.from_struct/1`, and question whether you do.

4.4 A fallback that hides missing implementations

```
defprotocol Billing.Price do
  @fallback_to_any true
  def amount(item)
end

defimpl Billing.Price, for: Any do
  def amount(_), do: 0
end
```

Every new product type without an implementation is now free, silently. The `Protocol.UndefinedError` you would have got in the first test is gone. Fix: no fallback for anything with business meaning. If a default is really correct, opt in per struct with `@derive Billing.Price`.

4.5 Unrelated behaviour in one multi-clause function

```
def update(%Product{} = p, attrs), do: Catalog.save(p, attrs)
def update(%Invoice{} = i, attrs), do: Accounting.amend!(i, attrs)
def update(%User{} = u, attrs), do: Accounts.change_email(u, attrs.email)
```

Three different operations, with different rules and failure modes, share a name. The design anti-patterns guide calls this *unrelated multi-clause function*: documentation, specs and callers can't describe it honestly. Fix: separate, named functions. If the operation really is the same idea across types, a protocol makes it explicit and extensible.

4.6 Choosing an implementation by building a module name

```
def notifier(kind) do
  Module.concat([MyApp.Notifiers, Macro.camelize(kind)])
end
notifier(params["channel"]).deliver(user, msg)
```

User input decides which module is called, atoms are created from input (chapter 1), and the compiler can't see the dependency. The macro anti-patterns guide calls this *untracked compile-time dependencies*. Fix: an explicit map from allowed values to modules:

```
@notifiers %{"email" => EmailNotifier, "sms" => SmsNotifier}
def notifier(kind), do: Map.fetch(@notifiers, kind)
```

4.7 Defining an implementation at runtime

```
# in a test helper or a script loaded after compilation
defimpl String.Chars, for: Money do
  def to_string(m), do: "#{m.amount} #{m.currency}"
end
```

Protocols are consolidated at build time. An implementation defined later is not part of the consolidated dispatch table, so it behaves differently in tests, `ix` and releases. Fix: keep every `defimpl` in `lib/`, so it is compiled with the project. For test-only structs, put them in `test/support` compiled via `elixirc_paths` in `mix.exs`.

5. Review checklist and sources

- ☐ Protocols are used for data-driven polymorphism; behaviours for chosen implementations
- ☐ Every callback is marked `@impl true`
- ☐ Structs with secrets derive `Inspect` with `only:` or `except:`
- ☐ No `Enum` functions on structs; iterate the intended field
- ☐ No `@fallback_to_any` for protocols with business meaning
- ☐ Multi-clause functions group one operation, not several unrelated ones
- ☐ Implementations are chosen from explicit maps, not modules built from strings
- ☐ All `defimpls` live in compiled project code

Sources

- Protocols: <https://hexdocs.pm/elixir/protocols.html>
- Typespecs and behaviours: <https://hexdocs.pm/elixir/typespecs.html>
- Inspect: <https://hexdocs.pm/elixir/Inspect.html>
- Design-related anti-patterns: <https://hexdocs.pm/elixir/design-anti-patterns.html>
- Meta-programming anti-patterns: <https://hexdocs.pm/elixir/macro-anti-patterns.html>